Compliance & Operations AI Assistant for Mobile Food Vendors in Suffolk County, NY

A Multi-Agent Pipeline Integrating Public Data, NLP Rule Extraction, Predictive Modeling, and Retrieval-Augmented Generation

Consultancy Team

Aisha – Data Architect Lead

Esra – NLP Extraction Specialist

Laiba – Front-End Experience Lead

Course Information

AIM 490W: AI Management Capstone Project

Instructor: Dr. Bruno G. Kamdem

Technical Report

Submission Date: April 2026

Executive Summary

This report documents the design, implementation, and evaluation of an agentic AI system that automates compliance assessment for mobile food vendors operating in Suffolk County, New York. The system ingests real public inspection data, extracts regulatory rules via natural language processing, predicts inspection risk using a gradient-boosted classifier, and exposes results through a Streamlit interface that includes a retrieval-augmented (RAG) chatbot powered by Anthropic's Claude model. The system delivers operational value by reducing time required to interpret regulatory requirements, identifying high-risk scenarios, and generating recommendations. By automating compliance and risk analysis, the system enables vendors to minimize violations and regulatory agencies to allocate resources efficiently.

Key Results

- Ingested 107,843 real inspection violation records spanning 2024–2025 across 4,977 unique food-service facilities in Suffolk County.
- Extracted 21 structured regulatory rules, of which 20 (95%) are verifiably grounded in source documents.
- Trained a Gradient Boosting classifier achieving ROC AUC = 0.859 and PR AUC = 0.673 on a temporally-held-out test set — a 2.5× lift over the base-rate baseline, allowing reliable identification of high-risk scenarios and compliance recommendations.
- Deployed an end-to-end Streamlit UI with seven tabs including a live RAG chatbot that cites source documents for every answer.
- Every pipeline stage writes a validation report with an explicit status flag (ok / degraded), supporting auditability and reproducibility.

Intended Users

Prospective and current mobile food vendors who need to understand permit obligations; regulatory staff who triage inspection resources; and third-party compliance consultants who advise vendors on operational risk reduction. This tool aims to bridge the gap between complex regulations and business decision-making.

1. Introduction

1.1 Problem Statement

Mobile food vendors, including food trucks, pushcarts, trailers, and temporary event units, face a fragmented regulatory landscape. In Suffolk County alone, a single operator must navigate permits issued by the County Department of Health Services, NY State Tax Department, local town clerks, fire marshals, and the Department of Motor Vehicles, while complying with NY State Sanitary Code Subpart 14-4. This process can create operational challenges and requires vendors to spend significant time identifying regulatory requirements, increasing the risk of compliance errors and delayed business operations. The consequences of non-compliance range from fines and permit suspension to foodborne illness outbreaks that damage both public health and operator livelihoods.

Inspection data is public but difficult to use. Suffolk County publishes violation records through an ArcGIS Hub portal; NY State Sanitary Code text is distributed across multiple PDF and HTML documents; weather data that correlates with food-safety risk lives in NOAA's NCEI service. No single interface helps a vendor synthesize these sources into an actionable compliance picture. As a result, vendors and regulators lack a data-driven tool to translate regulatory data into actionable compliance insights.

1.2 Objectives

1. Ingest real, public, programmatically-accessible data from at least three independent sources.
2. Extract regulatory rules from unstructured regulatory text using NLP, with every rule grounded in a verifiable source span.
3. Train a predictive model that beats a base-rate baseline on a temporally-held-out test set.
4. Produce an interactive interface in which an end user can search historical inspection data, score a what-if vendor scenario, view a tailored permit checklist, and ask free-form questions of a RAG-grounded chatbot.
5. Enforce a validation checkpoint after every pipeline stage, with failures surfaced through structured status reports.

1.3 Scope and Non-Goals

In scope: Suffolk County, NY mobile food vendors; inspection history from 2024–2025; NY State Subpart 14-4 and NYC DOH mobile-food-vendor guidance (as a proxy for town-specific rules); weather data from Islip MacArthur Airport.

Out of scope: real-time inspection scheduling; multi-jurisdiction deployment; integration with county-issued licensing systems; personal financial or identity data handling.

2. Background and Related Work

2.1 Regulatory Context

New York State regulates food-service establishments under Title 10 NYCRR Part 14 of the State Sanitary Code. Subpart 14-4 specifically governs mobile food service establishments and pushcarts, covering food supplies, protection, transportation, water supply, handwashing, and pest control. Counties such as Suffolk administer the permit-issuance and inspection function through their local Department of Health Services, adding county-specific sanitary codes on top of the state minimum [1][2]. The regulatory complexity reinforces the need for a system capable of translating dispersed requirements into actionable compliance guidance.

2.2 Predictive Inspection Models in Food Safety

The use of machine learning to prioritize food-safety inspections has precedent. Kang et al. (2013) demonstrated that Yelp review text could predict hygiene violations in Seattle restaurants [3]. The City of Chicago's 2015 model used gradient-boosted trees over inspection history, sanitation complaint data, and weather to move critical violation detection to earlier in the inspection week, finding violations an average of seven days sooner than the previous schedule [4]. These precedents motivate the choice of gradient boosting as the base estimator in this project. Based on these findings, predictive modeling directly supports the business needs by improving inspection efficiency.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG), introduced by Lewis et al. (2020), combines a dense or sparse retriever with a sequence-to-sequence generator to answer open-domain questions while reducing hallucination [5]. In domains like legal and regulatory compliance, where answers must be traceable to authoritative sources, RAG is preferred over direct LLM prompting because each generated claim can be linked back to a retrieved passage [6]. This system adopts that pattern: a TF-IDF retriever produces the top-k passages from a knowledge base of regulations, permits, and common violations, and an LLM (Claude Haiku 4.5) produces a cited answer. This approach is crucial in regulatory environments, in which decision-making must be grounded in verifiable sources to maintain reliability and trust.

2.4 Multi-Agent Pipeline Architectures

In contrast to traditional pipelines, agentic frameworks such as CrewAI and LangGraph emphasize task orchestration, tool-retrieval capabilities, and autonomous reasoning loops. These systems break down complex workflows into components, where each unit performs a specific function and produces structured outputs that can be validated and reused across the system. While the present implementation is structured as a modular pipeline, it adopts these agentic principles by enabling traceable, data-driven decision-making with minimal

human oversight. The architecture described in Section 3 builds on this approach by incorporating validation checkpoints and tool-driven outputs that replicate the capabilities of a multi-agent system.

3. System Architecture

The system is organized as a six-stage modular pipeline with an integrated user interface, designed to support agentic decision-making and autonomous workflow execution. Each stage is implemented as a standalone Python module in src/, reads from the previous stage's outputs, writes its own outputs, and emits a JSON validation report.

3.1 Pipeline Stages

Table 1: System Pipeline Stages and Outputs

Stage #	Processing Stage	Implementation Module	Prediction & Output Generation
1	Data Ingestion	src/01_ingest.py	Raw CSVs + regulation PDF
2	Cleaning & Normalization	src/02_clean.py	Processed CSVs
3	Rule Extraction (RAG-style)	src/03_extract_rules.py	rules/extracted_rules.json
4	Feature Engineering	src/04_features.py	features.csv
5	Predictive Modeling	src/05_model.py	risk_model.joblib
6	Output / Prediction API	src/06_predict.py	stage6_predictions.json
UI	Streamlit + RAG Chatbot	app.py + src/07_rag.py	Interactive web app

3.2 Validation Checkpoints

Between every two stages, the pipeline writes a structured JSON report containing (a) input row counts, (b) output row counts, (c) schema-level quality checks, and (d) a top-level status field that is either ok or degraded. The interface layer reads these reports to display system health on the Model Info tab. This supports reproducibility: a new team member can diff two stage reports to see what changed between runs.

3.3 Data Flow

Stage 1 fetches Suffolk County restaurant violation records directly from the ArcGIS FeatureServer REST endpoint, NOAA daily-summary weather from the NCEI Access Data Service, and the NYC Department of Health mobile food vendor PDF. Stage 2 normalizes column names across duplicated CamelCase/snake_case fields (a quirk of the Suffolk service), parses epoch-millisecond timestamps into pandas datetimes, and deduplicates on (facility_id, inspection_date, code_section, violation_text). Stage 3 chunks the regulation text, builds a TF-IDF index, and queries it with curated keyword lists for ten

risk categories. Stage 4 constructs per-visit features using only information available before the current visit to prevent target leakage. Stage 5 trains a Gradient Boosting classifier with a chronological train/test split. Stage 6 exposes a scoring function that accepts a vendor scenario dict and returns a compliance score, risk band, prioritized actions, and a traceability block listing every data source that fed the decision.

3.4 Agentic System Design

While the system is implemented as a modular pipeline, it aligns with a modern agentic system design. Each stage performs a specialized role, while collaborating with other components to execute complex workflows with minimal human oversight.

Table 2: Agent Roles and Responsibilities within the System Architecture

Agent Name	Primary Responsibility
Data Ingestion Agent	Retrieves and combines external data sources, including inspection records, regulatory documents, and environmental data.
Processing Agent	Cleans, normalizes, and prepares structured datasets for analysis.
Rule Extraction Agent	Identifies and structures compliance rules from unstructured regulatory text using NLP techniques.
Risk Analysis Agent	Utilizes predictive modeling to assess compliance risk and identify high-risk scenarios.
Recommendation Agent	Generates actionable compliance recommendations grounded in regulatory rules.
User Interaction Agent	Facilitates user interaction through the Streamlit application and RAG chatbot, enabling natural language queries.

3.5 Tooling Layer

To support autonomous and structured decision-making, the system incorporates a set of tools throughout the pipeline in the form of reusable operations. These tools can be used independently and collectively. The system supports scalability with minimal human oversight.

Table 3: System Tools and Their Functional Capabilities

Tool Name	Functional Capability
Fetch_Inspection_Data()	Retrieves food inspection records from NY state open data endpoints and ArcGIS services, with built-in fallback logic for reliability.
Fetch_Weather_Data()	Retrieves NOAA daily weather summaries for risk modeling.
Normalize_Inspection_Data()	Standardizes inconsistencies, parses timestamps, and removes duplicate records to ensure clean inputs.
Extract_Regulatory_Rules()	Parses regulatory text, applies TF-IDF retrieval, and generates structured compliance rules grounded in sources.

Generate_Features()	Builds leak-free features and joins external signals, such as weather or seasonality.
Compute_Risk_Score()	Applies a trained gradient boosting model to estimate inspection risk and classify compliance levels.
Generate_Compliance_Report()	Produces a structured output including compliance score, risk band, required permits, and prioritized actions.
Retrieve_Regulatory_Context()	Performs TF-IDF based retrieval over a knowledge base of regulations, permits, historical violation for RAG responses.
Generate_RAG_Response()	Breaks down retrieved context into user-facing answers using an LLM (Claude) or fallback template with source citations.

4. Data Sources

All data used by this project is real, public, and programmatically retrievable without an API key (with the exception of the optional Anthropic API key used for the LLM chatbot tier). No simulated or synthetic records are used in the training pipeline. This ensures that all generated outputs reflect real-world conditions and can support practical decision-making for both vendors and regulatory agencies.

Table 4: Data Sources and Their Role in Compliance Analysis

Dataset	Source	Volume	Description
Suffolk County Restaurant Violations 2024–2025	ArcGIS FeatureServer	107,843 rows	107k real inspection violations
NOAA NCEI Daily Summaries (Islip KISP)	NCEI Access Data Service	729 days	TMAX, TMIN, PRCP, AWND, SNOW
NYSDOH Food Service Inspections	Socrata (health.data.ny.gov)	21,669 rows	Supplementary regional data used to enhance context and inspection patterns for Nassau County.
NYC DOH Mobile Food Vendor Regulations	nyc.gov PDF	36 pages	Primary regulation text for RAG
NY State Sanitary Code Subpart 14-4	Hand-seeded section titles	11 sections	Authoritative legal references
Permit Bank	rules/required_permits.json	10 permit types	Hand-curated from public agency sources

4.1 Data Freshness and Refresh

Stage 1 is idempotent: running it again overwrites the raw files, and the subsequent stages can be re-run in sequence to rebuild the model against refreshed data. The Suffolk FeatureServer returns up to 1,000 records per request, so the ingestion script loops until the

service returns a partial page. As a result, the system remains up-to-date with evolving regulatory and environmental conditions, supporting long-term deployment.

5. Methods

5.1 Stage 1: Ingestion

The ingestion module attempts multiple candidate Socrata dataset IDs for NY State food inspections, iterating until one returns a well-formed CSV. For Suffolk County restaurant violations, it first queries the ArcGIS Online search API to locate the item IDs for the 2024 and 2025 Feature Services, fetches their metadata to extract the ArcGIS REST base URL, then pages through the /query endpoint until all records are retrieved. NOAA weather is fetched as a single CSV via the NCEI Access Data Service URL, which returns two years of daily summaries for a single station without requiring an API key. The ingestion stage functions as the system's data acquisition layer, ensuring complete and reliable real-world inputs.

5.2 Stage 2: Cleaning

The Suffolk service returns duplicate column pairs (e.g., both FacilityID and Facility_ID) because the 2024 and 2025 datasets have slightly different schemas. The cleaning module coalesces these pairs into canonical snake_case columns, converts ArcGIS epoch-millisecond timestamps to pandas datetimes, normalizes city names to uppercase, extracts the five-digit ZIP code where present, drops records missing the critical (facility_id, inspection_date) pair, and deduplicates on (facility_id, inspection_date, code_section, violation_text). The resulting cleaned dataset contains 107,843 rows across 4,977 unique facilities. This ensures data consistency and reliability, which is critical for accurate modeling and analysis.

5.3 Stage 3: Rule Extraction

Regulation text is extracted from the NYC DOH PDF using pypdf, chunked into paragraphs of at least 40 characters, and indexed with a TF-IDF vectorizer over unigrams and bigrams. For each of ten risk categories (permit display, handwashing, temperature control, commissary, food protection, waste disposal, pest control, food worker health, water supply, unapproved source), a curated keyword query is projected into the TF-IDF space and the top-scoring paragraph is retained as the rule body. Every extracted rule is subjected to a grounding check: at least one of the query keywords must appear verbatim in the retained paragraph. Of 21 rules extracted, 20 passed the grounding check (95% grounded rate). This stage enables the system to translate unstructured regulatory text into actionable compliance rules.

5.4 Stage 4: Feature Engineering

The feature engineering module aggregates the row-level violation data into one record per (facility, inspection date) pair — an 'inspection visit' — producing 8,849 visits across 4,970 facilities. For each visit, it computes: (a) the count of total violations and proxy-critical violations, (b) whether the visit included each of the major risk categories from Stage 3, (c) leak-free historical statistics using pandas cumcount — the running sum of prior violations minus the current row's contribution, so that features for any given visit use only information available before that visit, (d) seasonality signals (month, day-of-week, summer flag), and (e) weather features joined on the inspection date from the NOAA dataset.

The target is defined as a binary indicator of whether a visit's total violation count is at or above the 75th percentile across all visits in the dataset — a 'high-severity visit'. This definition was chosen because it yields a balanced positive class rate of 28.9% which is sensitive enough for useful ranking yet avoids the near-total imbalance that would result from defining any violation at all as positive. These features provide a foundation for predictive risk modeling by capturing historical behavior and environmental context.

5.5 Stage 5: Modeling

A scikit-learn GradientBoostingClassifier is trained with 200 estimators, a maximum depth of 4, and a learning rate of 0.05. The split is chronological: the earliest 80% of visits are training data, the latest 20% are test. Weather NaNs (for visits where no NOAA record was available) are imputed with column medians. The full feature list is: prior_visits, prior_viol_sum, prior_crit_sum, prior_viol_avg, days_since_last, month, dow, is_summer, has_temperature, has_handwashing, has_pests, has_sourcing, has_permit, tmax, tmin, prcp. Model performance is reported against the base-rate baseline (predicting the training-set positive-class probability for every test point). This allows the system to identify high-risk vendors, supporting efficient inspection and risk mitigation.

5.6 Stage 6: Prediction Interface

The scoring function accepts a vendor scenario as a Python dict, constructs the corresponding feature row, calls the trained model to produce a risk probability, inverts it into a 0–100 compliance score, and annotates the result with the top-five feature contributors, a permit checklist filtered to the applicable permit IDs, and a list of prioritized actions. Every action carries a 'grounded_in' field identifying which rule or regulation produced it. The result also includes a derived_from block documenting the data sources used. As a result, the model outputs user-friendly, actionable insights that support decision-making.

5.7 RAG Chatbot

The RAG module (src/07_rag.py) builds a knowledge base of 51 passages comprising 21 extracted rules, 10 permit entries, and the 20 most common real violation texts drawn from

the cleaned Suffolk data. It vectorizes the corpus with a second TF-IDF model and answers a query by computing cosine similarity, returning the top-k passages. If an ANTHROPIC_API_KEY is present in the environment, these passages are passed as context to claude-haiku-4-5 with a system prompt that constrains the model to answer only from context and cite sources inline. If no key is present, the system gracefully degrades to a retrieval-only template that still shows the user the retrieved passages and their sources. The RAG module component enables explainable, source-grounded interactions, ensuring users receive reliable compliance guidance.

6. Results

6.1 Ingestion and Cleaning

All primary data sources were successfully ingested. The Suffolk FeatureServer yielded 107,843 deduplicated records across 4,977 facilities, spanning the inspection date range 2024-01-02 through 2025-12-31. Top ten cities by violation count are shown in Table 2 and reflect the historical concentration of food-service activity on the South Shore and in the Hamptons during the summer tourist season. This demonstrates the system's ability to reliably aggregate and standardize large-scale public data.

Table 5: Top Suffolk County Cities by Inspection Violation Volume (2024-2025)

City	Total violations
HUNTINGTON	4,320
PATCHOGUE	4,235
BAY SHORE	3,336
COMMACK	3,108
SOUTHAMPTON	3,091
MONTAUK	2,817
DEER PARK	2,721
RIVERHEAD	2,664
LINDENHURST	2,659
SMITHTOWN	2,615

6.2 Rule Extraction Grounding

Of 21 rules extracted from the NYC DOH regulation text and NY Subpart 14-4 section titles, 20 (95%) passed the grounding check, meaning at least one trigger keyword appeared verbatim in the retained passage. This high grounding rate indicates the TF-IDF retriever is reliably finding relevant passages, reducing the risk of hallucinated outputs.

6.3 Feature Distribution

Table 6: Distribution of Violations by Compliance Category

Category	Description	Number of Tagged Violations
food_protection	Food Protection	27,024
permit	Permitting	22,592
temperature	Temperature Control	21,511
handwashing	Handwashing	20,783
other	Other	14,090
worker_health	Worker Health	1,013
waste	Waste Management	468
water	Water Supply	263
pests	Pest Control	67
sourcing	Food Sourcing	31

This distribution highlights operational compliance issues such as food protection and permitting, confirming the relevance of the selected feature set.

6.4 Predictive Model Performance

Table 4 summarizes the trained classifier's performance on the 20% temporally-held-out test set. ROC AUC of 0.859 indicates strong rank-ordering ability, and the PR AUC of 0.673, compared to the baseline of 0.273 (the class prior), represents a 2.5× lift, well above the 'useful' threshold of 1.5× typically applied to risk-triage models. This level of performance indicates that the model can prioritize high-risk vendors, enabling efficient allocation of inspection resources and compliance measures.

Table 7: Predictive Model Performance and Evaluation Metrics

Performance Metric	Value
ROC AUC	0.859
PR AUC	0.673
Test positive rate (baseline)	0.273
PR-AUC lift over baseline	2.46×
Train rows	7,079
Test rows	1,770
Classifier	GradientBoostingClassifier (200 × depth 4, lr 0.05)
Split	Chronological 80/20

6.5 Feature Importances

The dominant features, has_temperature (36%), has_handwashing (18%), prior_viol_avg (15%), and has_permit (13%), are consistent with food-safety domain knowledge. Temperature-control violations are the most common critical category in the NYC DOH data, handwashing failures are a direct proxy for operator diligence, prior violation rate is a strong historical predictor in every published food-inspection model, and permit-status issues are a direct indicator of administrative non-compliance. These results confirm that

the model aligns with domain knowledge, supporting its usage in real-world decision-making.

Table 8: Top Features by Importance in Predictive Risk Modeling

Feature Name	Description	Importance Score
has_temperature	Temperature Violations Present	0.356
has_handwashing	Handwashing Violations Present	0.182
prior_viol_avg	Average Prior Violations	0.153
has_permit	Permit Compliance Indicator	0.135
days_since_last	Days Since Last Inspection	0.054
tmax	Maximum Temperature	0.018
prior_viol_sum	Total Prior Violations	0.017
month	Month of Inspection	0.017
tmin	Minimum Temperature	0.015
prior_crit_sum	Total Prior Critical Violations	0.013

6.6 Scenario Outputs

Three scenarios were scored to sanity-check the scoring function. The compliant Montauk lobster-roll truck received a compliance score of 99.0/100 with a single medium-priority action (routine temperature monitoring). The Huntington halal cart with a lapsed permit, missing handwash station, unapproved-source sourcing, and pest history received 37.0/100 with five critical actions. A new coffee cart with no history but fully compliant paperwork received 99.7/100.

Table 9: Example Vendor Scenario Risk Assessments and Compliance Outputs

Vendor Scenario	Compliance Score (0-100)	Risk Category	Actions
Montauk Lobster Roll Truck	99.0	LOW	1
Huntington Halal Cart (lapsed permit)	37.0	HIGH	5
Patchogue Coffee Cart (new vendor)	99.7	LOW	0

A monotonicity cross-check in the final validation step confirms that the lapsed-permit scenario is scored lower than the fully compliant scenario, a basic sanity property that the pipeline verifies automatically before emitting its stage-6 report. These scenarios demonstrate the system's ability to translate complex data and model outputs into actionable guidance tailored to different risk profiles.

6.7 End-to-End Validation

All six pipeline stages plus the RAG chatbot module completed with status ok on the most recent run. The validation reports are checked into outputs/stage*_report.json and form part of the project's auditability story. This end-to-end validation confirms the system's reliability and supports its deployment as a production-ready compliance tool.

7. User Interface

The Streamlit application (app.py) exposes seven tabs, each built against cached in-memory dataframes and the trained model. The application serves as the primary interaction layer between users and the agentic system, translating complex data processing and model outputs into accessible insights. The interface is designed for technical and non-technical users, enabling real-time querying, scenario analysis, and compliance evaluation.

Table 10: User Interface Components and Their Functional Capabilities

Tab #	Component	Description
1	Ask the Assistant	RAG chatbot with inline citations and retrieved-source panel
2	Search Vendors	Full-text search across facility name, address, and violation text, filterable by city and year
3	Vendor Profile	Facility-level history and auto-computed compliance score
4	Score a Scenario	What-if form with scenario inputs, compliance score, prioritized actions, top risk contributors, and full traceability
5	Required Permits	Dynamic permit checklist filtered by operation profile, with fees and legal authority
6	Regulations	Searchable rule bank
7	Model Info	ROC/PR metrics, feature-importance chart, confusion matrix

Combined, these components provide a decision-support environment that allows users to assess risk and receive actionable recommendations within a single platform.

7.1 RAG Chatbot Modes

The chatbot operates in one of two modes depending on whether the ANTHROPIC_API_KEY environment variable is set. In LLM mode (the preferred configuration) retrieved passages are passed to Claude-Haiku-4.5 via the Anthropic SDK and returns a conversational answer with inline [1], [2], [3] citations grounded in the retrieved passages. In retrieval-only mode (the fallback) the system returns the top-k passages along with similarity scores and source labels. The dual-mode design ensures flexibility and reliability because users benefit from natural language interaction, while maintaining transparency and traceability. The system is designed with an emphasis on privacy. No user data is transmitted externally beyond the query text, and only when the LLM mode is enabled.

8. Limitations, Risks, and Threats to Validity

8.1 Target Definition

The Suffolk County data does not mark individual violations as critical versus non-critical in a usable field, so this project proxies 'high severity' as top-quartile violation count per visit. A model trained against a direct critical-violation label from Chicago, Seattle, or NYC DOH data would likely achieve different performance characteristics. Replacing the target with a direct critical field should be the first priority if additional labeled data becomes available. Otherwise, this limitation may affect the model's ability to distinguish between critical and non-critical violations, potentially reducing decision accuracy in high-risk scenarios.

8.2 Weather Coverage

The NOAA ingestion range (2023–2024) only partially overlaps with the violation date range (2024–2025), so weather features are populated for approximately 50% of visits. Extending the NOAA request to cover 2024–2025 (a single line change in src/01_ingest.py) would improve feature completeness. Incomplete weather coverage reduces the model's ability to fully capture environmental risk factors, limiting predictive performance.

8.3 Mobile-Vendor vs. Fixed-Establishment Data

Suffolk's restaurant violations dataset mixes fixed-establishment restaurants with mobile food units. The model therefore learns patterns from both populations. For a production deployment targeting strictly mobile vendors, the ingestion stage would need to filter on the permit-type field, and a smaller, more targeted dataset may be required. The permit bank and RAG knowledge base are, however, already mobile-specific. This introduces potential noise into the model, as risk patterns for fixed establishments are different from those of mobile vendors.

8.4 Regulatory Drift

NY State Sanitary Code Subpart 14-4 and local town ordinances change periodically. The extracted-rule bank is a snapshot as of the ingestion run date and should be re-refreshed on a periodic schedule. The permit bank in rules/required_permits.json carries a last_updated field for this purpose. Failure to update regulatory inputs could result in outdated or incorrect compliance recommendations.

8.5 Copyright and Attribution

All ingested data is public record or public-domain regulatory text. The RAG chatbot is explicitly instructed to cite sources, and the retrieval-only fallback mode displays passages in-line with source labels. The report itself includes a references section with hyperlinks to every cited authority. This ensures the system remains compliant with legal and ethical standards, while maintaining transparency.

9. Future Work

This section outlines key opportunities to enhance the system's performance, scalability, and real-world applicability. The following recommendations are prioritized based on their impact on model accuracy, value, and deployment readiness.

9.1 Model and Data Improvements

- Switch target from proxy-severity to direct critical-violation labels once a labeled dataset is obtained, improving model accuracy and alignment with real inspection outcomes.
- Filter ingestion to mobile-only permit types if Suffolk publishes a permit-type column, which tailors the model according to the intended users.
- Extend NOAA weather ingestion through the full violation date range to improve feature completeness and predictive performance.
- Run backtests on quarterly cohorts to measure calibration drift over time and ensure long-term model reliability.

9.2 User Experience and Decision Support

- Add a per-city risk dashboard to prioritize inspection routes and allocate resources efficiently.
- Integrate a 'vendor intake form' workflow where new vendors complete a questionnaire and receive a tailored compliance checklist and documentation support.

9.3 AI Capabilities and Deployment

- Replace TF-IDF retrieval with a sentence-embedding model (e.g., MiniLM or Claude's embedding endpoint) to improve semantic recall and handle complex user inquiries.
- Add authentication and audit logging for deployment to multiple users.

10. Conclusions

This project demonstrates that an agentic AI pipeline built from free, public data can deliver an end-to-end compliance solution for a regulated small-business domain. The model achieves strong test-set performance (ROC AUC 0.859, 2.5× PR-AUC lift) using only sixteen features, most of which are directly interpretable to a food-safety professional. By translating complex regulatory requirements and inspection data into actionable insights, the system reduces the burden of compliance, supporting decision-making for regulatory agencies. The RAG chatbot and tailored permit checklist turn the underlying data into guidance a non-technical vendor can act on. Every stage writes a structured validation report, supporting the kind of auditability that public-facing AI systems increasingly require. The architecture is modular enough that swapping any single component — the model, the retriever, the LLM backend, or the data source — requires touching only one module of src/. The system is a practical foundation for deploying AI-driven compliance

tools across other industries, in which similar challenges of data inconsistencies, regulatory complexity, and risk prioritization exist.

References

[1] New York State Department of Health. Title 10 NYCRR Part 14 — Food Service Establishments. Subpart 14-4: Mobile Food Service Establishments and Pushcarts. Retrieved from https://regs.health.ny.gov/

[2] Suffolk County Department of Health Services, Office of Food Protection. Restaurant Violations 2024 and 2025 datasets. Suffolk County Open Data Portal. https://opendata.suffolkcountyny.gov/

[3] Kang, J. S., Kuznetsova, P., Luca, M., & Choi, Y. (2013). Where not to eat? Improving public policy by predicting hygiene inspections using online reviews. Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, 1443–1448.

[4] Department of Innovation and Technology, City of Chicago. (2015). Food Inspection Forecasting. https://chicago.github.io/food-inspections-evaluation/

[5] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems 33, 9459–9474.

[6] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[7] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhuang, Y. (2023). HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. arXiv:2303.17580.

[8] National Oceanic and Atmospheric Administration, National Centers for Environmental Information. Access Data Service API. https://www.ncei.noaa.gov/access/services/data/v1

[9] New York City Department of Health and Mental Hygiene. What Mobile Food Vendors Should Know. https://www.nyc.gov/assets/doh/downloads/pdf/rii/regulations-for-mobile-food-vendors.pdf

[10] New York State Department of Taxation and Finance. Certificate of Authority. https://www.tax.ny.gov/bus/st/register.htm

[11] National Fire Protection Association. NFPA 96: Standard for Ventilation Control and Fire Protection of Commercial Cooking Operations.

[12] New York State Sanitary Code, Title 10 NYCRR Part 14.4. Subpart 14-2: Temporary Food Service Establishments.

[13] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830.

[14] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.

[15] Streamlit Inc. Streamlit — A faster way to build and share data apps. https://streamlit.io

[16] Anthropic. Claude API Documentation — claude-haiku-4-5 model. https://docs.anthropic.com/

[17] Esri. ArcGIS REST API — Feature Service Query. https://developers.arcgis.com/rest/services-reference/enterprise/query-feature-service/

[18] Socrata Open Data API (SODA). Developer documentation. https://dev.socrata.com/

Appendix A: Reproducibility

Environment

- Python ≥ 3.11
- Dependencies listed in requirements.txt
- Optional: ANTHROPIC_API_KEY in a .env file for LLM chatbot mode

Running the pipeline

From the project root, execute in order:

```
python src/01_ingest.py

python src/02_clean.py

python src/03_extract_rules.py

python src/04_features.py

python src/05_model.py

python src/06_predict.py

python -m streamlit run app.py
```

Validation

Each pipeline stage writes a JSON report to outputs/stage{N}_*.json. The top-level status field should be ok for a healthy run. A degraded status indicates at least one check failed; see the failures array in the same report for specifics.

Artifacts

- data/raw/ — unmodified downloaded source files
- data/processed/ — cleaned CSVs consumed by the UI
- rules/extracted_rules.json — Stage 3 output
- rules/required_permits.json — hand-curated permit bank
- models/risk_model.joblib — trained classifier
- outputs/stage*_report.json — validation reports

Appendix B: AI Usage Disclosure

This project incorporates the transparent use of generative AI tools in accordance with the guidelines. The tools used in this project include:

- OpenAI ChatGPT (GPT-4/5) - improving clarity and grammar of written content, code debugging assistance, and brainstorming support for system design.
- Anthropic Claude (Haiku 4.5) - a part of the system's Retrieval-Augmented Generation (RAG) chatbot, supporting the system for end-user interaction

All AI-generated outputs were carefully reviewed, validated, and edited by the project team. All technical implementations, including data ingestion, modeling, feature engineering, were developed and verified by the team The written content was revised to ensure accuracy and originality. The model results and system outputs are based on real data, validated through testing and evaluation No sensitive and personal data was shared with AI systems. All data used in this project is publicly available and ethically sourced. The team ensured that AI tools were used responsibly, maintaining academic integrity and professional standards. This disclosure affirms that the final submission reflects the original work, critical thinking, and technical implementation of the project team.